

LAB 6

Inheritance in C++

Single Inheritance, Access Specifiers, and Constructor Chaining

Learning Objectives

By the end of this lab, you should be able to:

- Explain what inheritance is and why it is one of the four pillars of OOP.
- Identify the base class and the derived class in any inheritance relationship.
- Write a derived class in C++ using the class Derived : public Base syntax.
- Call a base-class constructor from a derived-class constructor (constructor chaining).
- Use inherited member functions from inside a derived class.
- Explain the difference between public, protected, and private members.
- Predict what becomes accessible (and what does not) when you use public vs protected inheritance.

1. Why Do We Need Inheritance?

Imagine you have already written a Vehicle class. Now you need a Car class. A car IS a vehicle — it has a speed, it can move, it has all the properties of a vehicle plus a few extras (brand, model, number of seats). Without inheritance, you would have to copy the entire Vehicle class into Car and add the new bits — and again into Truck, Bike, Bus, etc.

Inheritance is the OOP feature that lets one class automatically gain the data and behavior of another class. The new class is allowed to reuse, extend, and customize what the original class already provides.

Three benefits inheritance gives you

Code reusability — write common functionality once in the base class; every derived class gets it for free.

Logical hierarchy — code reflects real-world relationships (a Car IS a Vehicle, a Student IS a Person).

Maintainability — fix or improve the base class once, and every derived class benefits automatically.

2. Single Inheritance — The Basics

Single inheritance means a derived class inherits from exactly one base class. That is the simplest, most common form, and the only form we cover in this lab.

Terminology

- **Base class** (also called parent or superclass) — the class being inherited from.
- **Derived class** (also called child or subclass) — the class that inherits.
- **Access specifier** — the keyword (public, protected, or private) that controls how base members appear in the derived class.

The Syntax

```
class Base {  
    // base class members  
};  
  
class Derived : access_specifier Base {  
    // derived class members  
};
```



Read it like English

class Car : public Vehicle → "class Car, which publicly inherits from Vehicle."

3. Code 1 — Basic Single Inheritance (Vehicle → Car)

Goal: build the simplest possible inheritance — a Vehicle base class and a Car class that derives from it. We also see how the derived constructor calls the base constructor.

The Code

```
#include <iostream>
using namespace std;

class Vehicle {
protected:
    int speed;
public:
    Vehicle(int s) {
        speed = s;
        cout << "Vehicle constructor called" << endl;
    }
    void showSpeed() {
        cout << "Speed: " << speed << endl;
    }
};

class Car : public Vehicle {
private:
    string brand;
public:
    Car(int s, string b) : Vehicle(s) {    // calling base constructor
        brand = b;
        cout << "Car constructor called" << endl;
    }
    void display() {
        showSpeed();                      // inherited function
        cout << "Brand: " << brand << endl;
    }
};

int main() {
    Car c(120, "Toyota");
    c.display();
}
```

Code Breakdown

Code	What it does
class Vehicle {	← Define the base class.
protected: int speed;	← speed is protected — accessible inside Vehicle AND inside any class that inherits from it, but NOT from main.

Code	What it does
<code>Vehicle(int s) { ... }</code>	← Base-class constructor — takes a speed and stores it.
<code>class Car : public Vehicle {</code>	← Car publicly inherits from Vehicle. Car automatically has speed and showSpeed().
<code>Car(int s, string b) : Vehicle(s)</code>	← The <code>: Vehicle(s)</code> part is constructor chaining — it tells C++ to run the Vehicle constructor first, passing <code>s</code> to it.
<code>showSpeed();</code>	← Called from inside Car — works because Car inherited it from Vehicle.
<code>Car c(120, "Toyota");</code>	← Creates a Car. Triggers Vehicle's constructor first, then Car's constructor.

Expected Output

► Output

```
Vehicle constructor called
Car constructor called
Speed: 120
Brand: Toyota
```

📦 Construction order is base → derived

C++ guarantees that the base-class constructor runs BEFORE the derived-class constructor body. This makes sense: a Car cannot finish being built until the Vehicle part of it exists.

⚠️ What if you forget the `: Vehicle(s)`?

If you omit the base-class initializer, C++ tries to call Vehicle's default (no-argument) constructor. Since Vehicle does not HAVE a no-argument constructor in this example, you get a compile error. Either provide one in the base class, or always chain to an existing constructor.

4. Code 2 — Person → Student (Reusing a Base-Class Function)

Goal: a slightly richer example. A Student IS a Person, plus an id and a cgpa. The Student class also reuses Person's display function instead of rewriting it.

The Code

```
#include <iostream>
using namespace std;

// Base Class
class Person {
protected:
    string name;
    int age;
public:
    Person(string n, int a) {
        name = n;
        age = a;
        cout << "Person Constructor Called" << endl;
    }
    void displayPerson() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};

// Derived Class
class Student : public Person {
    int id;
    float cgpa;
public:
    Student(string n, int a, int i, float c) : Person(n, a) {
        id = i;
        cgpa = c;
        cout << "Student Constructor Called" << endl;
    }
    void displayStudent() {
        displayPerson(); // calling base-class function
        cout << "ID: " << id << endl;
        cout << "CGPA: " << cgpa << endl;
    }
};

int main() {
    Student s1("Rahim", 21, 101, 3.85);
    cout << endl;
    s1.displayStudent();
}
```

Code Breakdown

Code	What it does
<code>protected: string name; int age;</code>	← Both are protected, so Student can access them directly (a private member could not be touched even by Student).
<code>Person(string n, int a)</code>	← Base constructor — stores name and age.
<code>class Student : public Person</code>	← Student inherits everything public/protected from Person.
<code>Student(...) : Person(n, a)</code>	← Forward name and age to the base-class constructor so it can do its job.
<code>id = i; cgpa = c;</code>	← Initialize Student's OWN extra fields.
<code>displayPerson();</code>	← Reusing the base class function — no need to rewrite the Name/Age printing logic.
<code>Student s1("Rahim", 21, 101, 3.85);</code>	← Triggers Person constructor first, then Student constructor.

Expected Output

► Output

Person Constructor Called
Student Constructor Called

Name: Rahim
Age: 21
ID: 101
CGPA: 3.85

💡 Why reuse `displayPerson()` instead of rewriting it?

If you later change how Person formats its output (e.g., adding a title), Student picks up the change automatically. Copying code = two places to fix = bugs. Inheritance = one place to fix.

5. Access Specifiers — A Quick Refresher

Before the next example, recall what the three access specifiers actually mean. They control WHO is allowed to touch a member.

Specifier	Accessible inside the class itself?	Accessible inside derived classes?	Accessible from outside (main)?
public	Yes	Yes	Yes
protected	Yes	Yes	No
private	Yes	No	No

The key distinction

protected is the inheritance-friendly version of private. Use it when you want derived classes to access the member, but you still want to hide it from outside code.

6. Code 3 — Access Specifiers in Inheritance (A → B, C)

Goal: see how the inheritance mode (public vs protected) further restricts what derived classes inherit. This example is famous for being tricky — read the comments carefully.

The Code

```
#include <iostream>
using namespace std;

class A {
public:
    int x;          // Public member
protected:
    int y;          // Protected member
private:
    int z;          // Private member
public:
    void setValues() {
        x = 10;
        y = 20;
        z = 30;
    }
    void showA() {
        cout << "Inside Class A:\n";
        cout << "x = " << x << ", y = " << y << ", z = " << z << endl;
    }
};
```

```

// Public Inheritance – A's public stays public, A's protected stays protected
class B : public A {
public:
    void showB() {
        cout << "\nInside Class B (Public Inheritance):\n";
        cout << "x = " << x << endl;    // Accessible
        cout << "y = " << y << endl;    // Accessible
        // cout << "z = " << z;        // ERROR – z is private in A
    }
};

// Protected Inheritance – A's public BECOMES protected in C
class C : protected A {
public:
    void showC() {
        cout << "\nInside Class C (Protected Inheritance):\n";
        cout << "x = " << x << endl;    // Accessible (but now protected)
        cout << "y = " << y << endl;    // Accessible
        // cout << "z = " << z;        // ERROR – z is private in A
    }
};

int main() {
    B obj1;
    obj1.setValues();    // OK – setValues is public in B
    obj1.showA();
    obj1.showB();

    C obj2;
    // obj2.setValues();    // ERROR – setValues became protected in C
    // obj2.showC();        // OK to call, but x and y are uninitialized

    cout << "\nAccessing from main():\n";
    cout << obj1.x << endl;    // OK – public
    // cout << obj1.y;        // ERROR – y is protected
    // cout << obj1.z;        // ERROR – z is private
    // cout << obj2.x;        // ERROR – x is protected in C

    return 0;
}

```

⚠ The commented-out lines are NOT optional — they are the lesson

Each commented line shows you exactly what would fail to compile and WHY. Uncomment them one at a time to see the compiler error for yourself; that is the whole point of this exercise.

7. Access Behavior — The Reference Tables

Inside Derived Classes

Member from A	In Class B (public)	In Class C (protected)
x (public)	Accessible (public)	Accessible (protected)
y (protected)	Accessible (protected)	Accessible (protected)
z (private)	Not Accessible	Not Accessible

From main (Outside Access)

Object	x	y	z
obj1 (B)	Yes	No	No
obj2 (C)	No	No	No

Plain-English Summary of What Happens

- **private members are never inherited usably.** z exists in B and C in memory, but neither class can touch it directly. Only A itself can.
- **public inheritance preserves access levels.** Public stays public, protected stays protected. This is the most common form — "is-a" relationships.
- **protected inheritance demotes public to protected.** Inside C, x and y are both protected — so main cannot use obj2.x even though x is public in A.
- **If a function becomes protected, you cannot call it from main.** That's why obj2.setValues() fails — setValues was public in A, but protected inheritance demoted it.
- **If you never call setValues, the variables are uninitialized.** Reading them will print garbage. This is a great reminder to always initialize your data.

8. Summary

Five things to take away from this lab:

- **Inheritance models "is-a" relationships.** A Car IS a Vehicle. A Student IS a Person.
- **Use protected for fields you want shared with subclasses** but hidden from outside code.
- **Chain constructors with : Base(args).** Base is built first, derived second.
- **private members are never directly accessible in a derived class.** Use protected or expose them through getters/setters.
- **The inheritance mode (public/protected/private)** further restricts access. public inheritance is the default choice 95% of the time.

9. Practice Tasks

Task 1 — Employee → Manager

Create the following:

- **Base class:** Employee — has *name* and *id*, with a constructor and a display function.
- **Derived class:** Manager — adds *salary* and *department*. Constructor should chain to Employee. Provide a function to display everything.



Hints for Task 1

Use protected for name and id so Manager can read them directly.

In Manager's display function, reuse Employee's display function instead of re-printing name and id.

Test by creating at least two Manager objects in main with different departments.

Task 2 — Shape → Rectangle

Create the following:

- **Base class:** Shape — with a function display() that prints something like "This is a shape."
- **Derived class:** Rectangle — stores *length* and *width*, and has a function area() that returns length × width.



Hints for Task 2

Inside Rectangle, call the base-class display() first, then print the area on the next line.

Try changing the inheritance mode from public to protected and observe what stops working in main.

Task 3 — Account → SavingsAccount (private members & accessors)

Create the following:

- **Base class:** Account — has *balance* as a **private** member (NOT protected). Provide *getBalance()* and *setBalance()* functions.
- **Derived class:** SavingsAccount — adds *interestRate* and a function *addInterest()* that updates the balance.

 What you MUST observe first

Inside SavingsAccount, first try to write *balance = balance + interest;* directly. You will get a compile error because balance is private in Account. **Read the error carefully** — this is the whole lesson.

 Then fix it the right way

Use the inherited *getBalance()* and *setBalance()* functions instead. For example:

```
setBalance(getBalance() + getBalance() * interestRate);
```

Test by creating a SavingsAccount with an initial balance, calling *addInterest()* a few times, and printing the new balance.

Prepared by

Alifa Shanzidah Manarat

Lecturer

Dept. of CSE, BAUST